

Statistical and Machine Learning SS26

Exercise Sheet 6 (incl. solution)

Classification tree

Goal: Use classification trees to predict whether car seat sales are “High” or “Low”

The `Carseats` data set contains sales of child car seats at 400 different stores

`Sales` is a continuous variable (Unit sales (in thousands) at each location), we want to convert it into a binary variable

```
library(tree) #used to construct classification and regression trees
library(ISLR2) #contains carseats data set
attach(Carseats)
str(Carseats)
```

```
## 'data.frame': 400 obs. of 11 variables:
## $ Sales : num 9.5 11.22 10.06 7.4 4.15 ...
## $ CompPrice : num 138 111 113 117 141 124 115 136 132 132 ...
## $ Income : num 73 48 35 100 64 113 105 81 110 113 ...
## $ Advertising: num 11 16 10 4 3 13 0 15 0 0 ...
## $ Population : num 276 260 269 466 340 501 45 425 108 131 ...
## $ Price : num 120 83 80 97 128 72 108 120 124 124 ...
## $ ShelfLoc : Factor w/ 3 levels "Bad","Good","Medium": 1 2 3 3 1 1 3 2 3 3 ...
## $ Age : num 42 65 59 55 38 78 71 67 76 76 ...
## $ Education : num 17 10 12 14 13 16 15 10 10 17 ...
## $ Urban : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 1 2 2 1 1 ...
## $ US : Factor w/ 2 levels "No","Yes": 2 2 2 2 1 2 1 2 1 2 ...
```

```
High <- factor(ifelse(Sales <= 8, "No", "Yes"))
# generate factor variable which takes on `yes` if the Sales variable exceeds 8, and `no` otherwise

# merge factor variable `High` with the rest of the Carseats data
Carseats <- data.frame(Carseats , High)
```

We now use the `tree()` function to fit a classification tree in order to predict `High` using all variables but `Sales`. The syntax of the `tree()` function is quite similar to that of the `lm()` function.

```
# fit model: dot to use all variables except sales as predictor
tree.carseats <- tree(High ~.-Sales, Carseats) # high was derived from Sales, we
#therefore exclude it because it doesn't contain new information

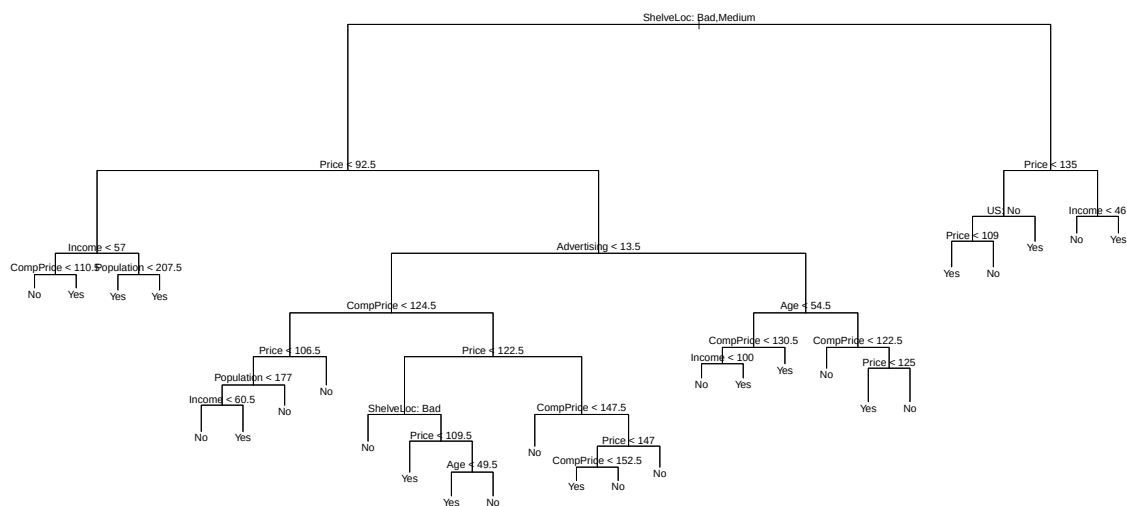
# Check internal nodes in the tree, the number of terminal nodes, and the (training) error rate
summary(tree.carseats)
```

```
##
## Classification tree:
## tree(formula = High ~ . - Sales, data = Carseats)
## Variables actually used in tree construction:
## [1] "ShelveLoc" "Price" "Income" "CompPrice" "Population"
## [6] "Advertising" "Age" "US"
## Number of terminal nodes: 27
## Residual mean deviance: 0.4575 = 170.7 / 373
## Misclassification error rate: 0.09 = 36 / 400
```

shows us variables actually used (selected most informative 8):
Variables such as Education apparently do not contribute to predicting whether sales are High or Low
the tree has 27 leaves: these are the endpoints where a final prediction ("Yes"/"No") is made.
training error rate is 9 % (36 out of 400 were misclassified). sounds good -
but we need to be skeptical as this model was tested on the same data it was trained on.
it has seen the answers already, so of course it knows what to do

One of the most attractive properties of trees is that they can be graphically displayed. We use the `plot()` function to display the tree structure, and the `text()` function to display the node labels.

```
plot(tree.carseats)
text(tree.carseats , pretty = 0)
```



pretty instructs R to include the category names for any qualitative predictors,
rather than simply displaying a letter for each category.
#left branch: yes, right branch: no

In order to properly evaluate the performance of a classification tree on these data, we must estimate the test error rather than simply computing the training error. We split the observations into a training set and a test set, build the tree using the training set, and evaluate its performance on the test data. The `predict()` function can be used for this purpose. In the case of a classification tree, the argument `type = "class"` instructs R to return the actual class prediction.

```

set.seed(123)
train <- sample(1:nrow(Carseats), 200)
Carseats.test <- Carseats[-train,]
High.test <- High[-train]
tree.carseats <- tree(High ~ .-Sales, Carseats, subset = train)
tree.pred <- predict(tree.carseats, Carseats.test, type = "class")
table(tree.pred, High.test) #confusion matrix

```

```

##           High.test
## tree.pred No  Yes
##           No  87  13
##           Yes 35  65

```

```

(87 + 65) / 200

```

```

## [1] 0.76

```

```

# correctly predicted no plus correctly predicted yes divided by total test obs (400 - 200)
# correct predictions for around 76 % of the locations in the test data set

```

Next, we consider whether pruning the tree might lead to improved results. The function `cv.tree()` performs cross-validation in order to determine the optimal level of tree complexity; cost complexity pruning `cv.tree()` is used in order to select a sequence of trees for consideration.

```

set.seed(123)
cv.carseats <- cv.tree(tree.carseats, FUN = prune.misclass)
# FUN = indicate that we want the classification error rate to guide the cross-validation
names(cv.carseats)

```

```

## [1] "size"    "dev"     "k"       "method"

```

```

cv.carseats

```

```

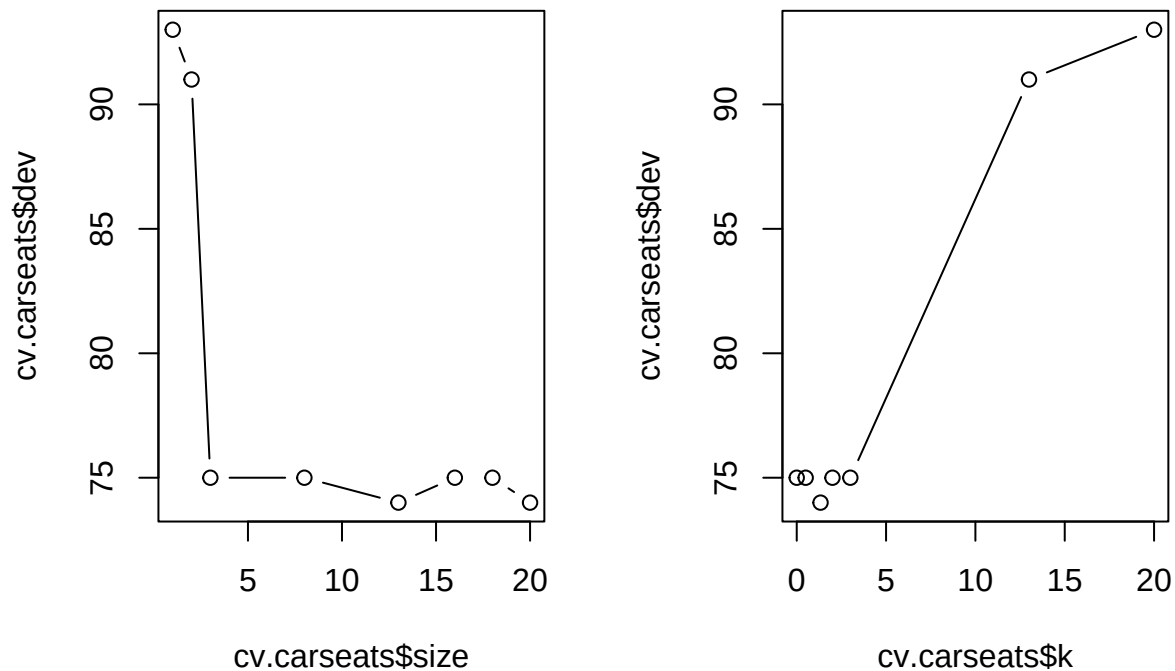
## $size
## [1] 20 18 16 13  8  3  2  1
##
## $dev
## [1] 74 75 75 74 75 75 91 93
##
## $k
## [1]      -Inf  0.000000  0.500000  1.333333  2.000000  3.000000 13.000000
## [8] 20.000000
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"          "tree.sequence"

```

```
#size: number of terminal nodes at each step (1: stump that predicts the same for all stores)
#dev: the cross-validation error at each size
#k: the complexity parameter (penalty for tree size, like lambda in regularization)
#method: just confirms "misclass" was used
```

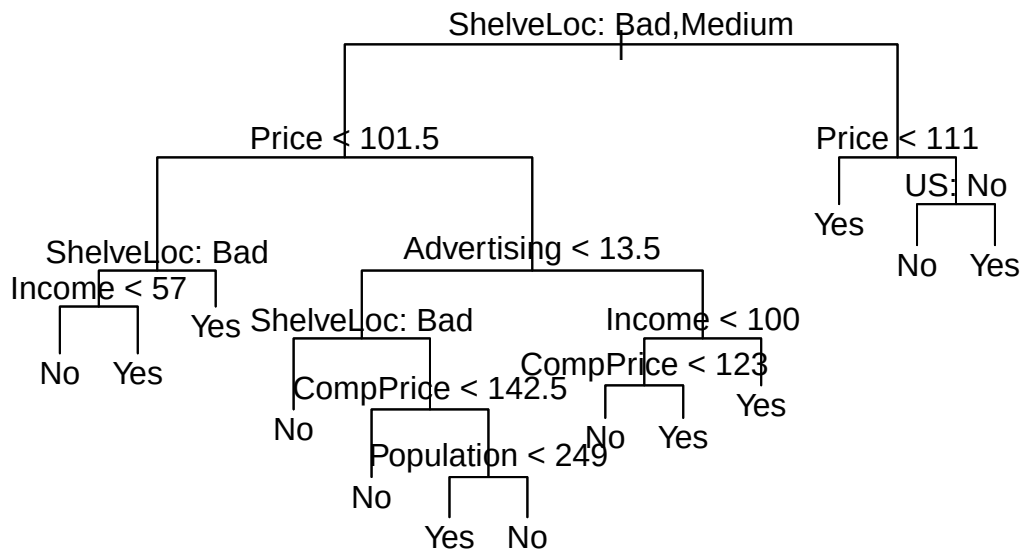
We plot the error rate as a function of both size and k. And we notice: the error barely changes between sizes 20 and 3. What does that suggest to you about whether you actually need a complex 20-node tree? K vs. error plot: as we increase k (decrease size), the tree gets forced to be simpler and simpler, and eventually too simple → error rises

```
par(mfrow = c(1, 2))
plot(cv.carseats$size , cv.carseats$dev,type ="b")
plot(cv.carseats$k, cv.carseats$dev,type ="b")
```



Prune the tree to obtain the nine-node tree.

```
prune.carseats <- prune.misclass(tree.carseats , best = 9)
#The error doesn't improve beyond size 9, so there's no benefit to keeping the extra nodes
plot(prune.carseats)
text(prune.carseats , pretty =0)
```



```
tree.pred <- predict(prune.carseats , Carseats.test , type = "class")
table(tree.pred , High.test)
```

```
##           High.test
## tree.pred No  Yes
##       No   86  13
##       Yes  36  65
```

```
(86 + 65) / 200
```

```
## [1] 0.755
```

```
# Now 77.5% of the test observations are correctly classified
# If we increase the value of `best`,
# we obtain a larger pruned tree with lower classification accuracy
```

Fit Regression Tree

We fit a regression tree to the `Boston` data set. First, we create a training set, and fit the tree to the training data

```

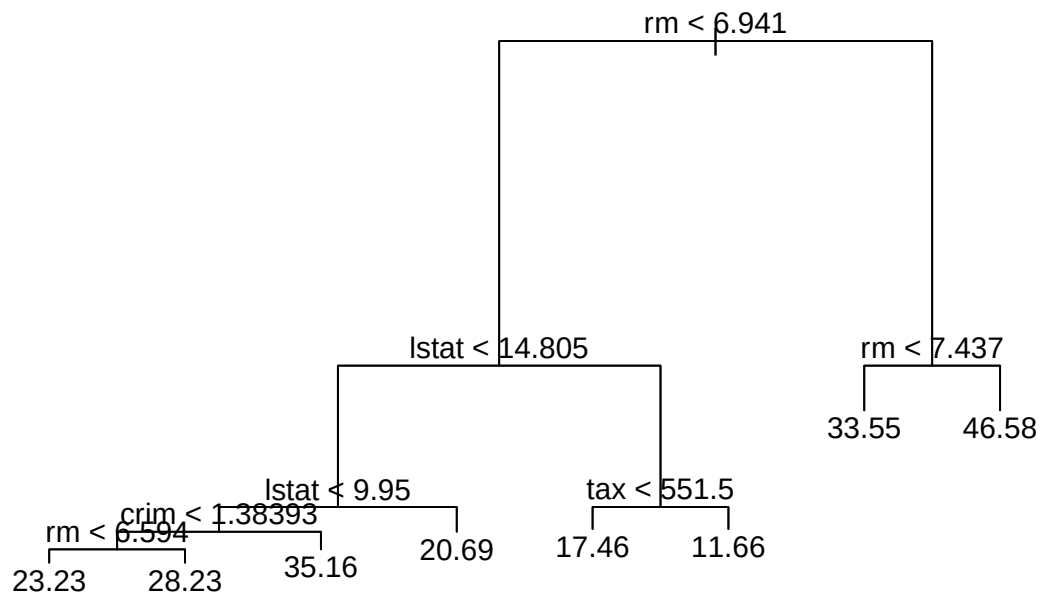
set.seed(123)
train <- sample(1:nrow(Boston), nrow(Boston) /2)
#Creates a training set by randomly sampling half the observations
tree.boston <- tree(medv ~ ., Boston, subset =train)
#fits regression (not classification) tree to predict meadian house value medv
# using all other variables, but only on training split
summary(tree.boston)

##
## Regression tree:
## tree(formula = medv ~ ., data = Boston, subset = train)
## Variables actually used in tree construction:
## [1] "rm" "lstat" "crim" "tax"
## Number of terminal nodes: 8
## Residual mean deviance: 12.98 = 3179 / 245
## Distribution of residuals:
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## -11.86000 -1.99500  0.03714  0.00000  1.86800  16.45000

# instead of misclassification error rate we now see Residual mean deviance
# That's because this is a regression tree predicting a continuous value (medv),
# not a category. We measure how far off our predictions are from the actual values

plot(tree.boston)
text(tree.boston, pretty =0)

```



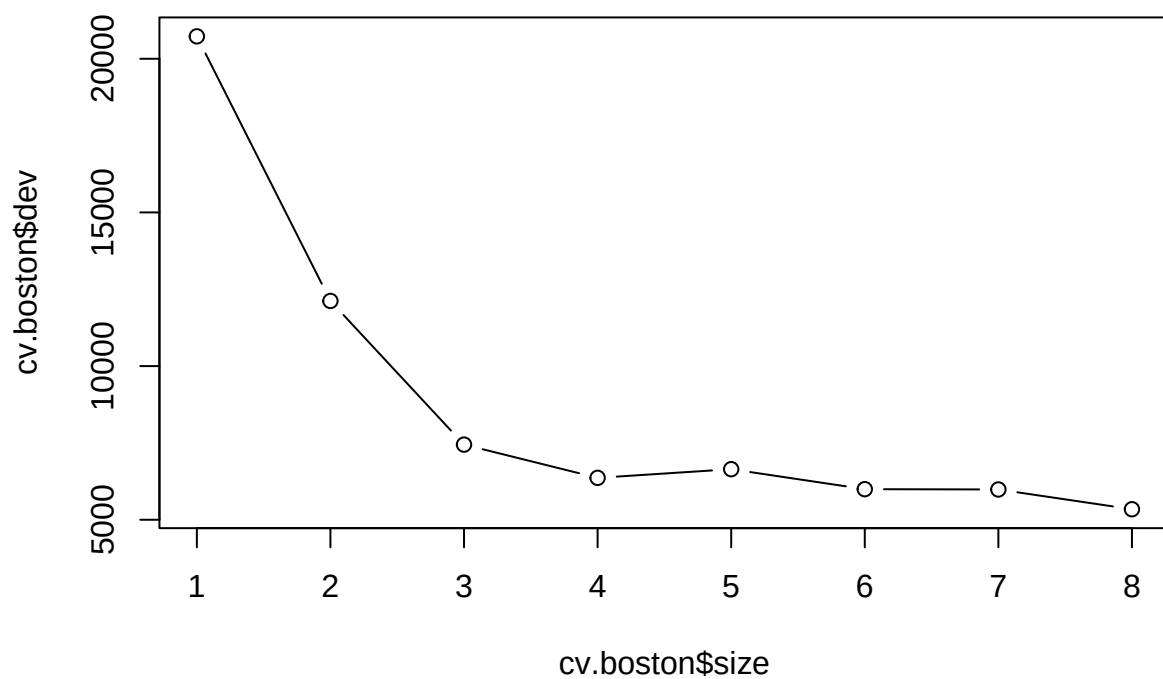
larger values of rm, or lower values of lstat, correspond to more expensive houses.

Now pruning the tree.

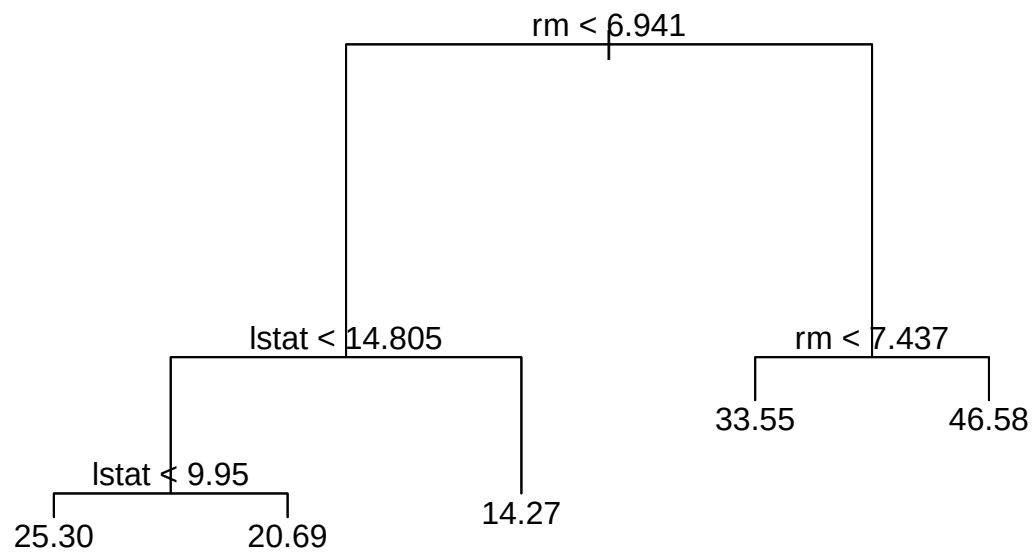
```

set.seed(123)
cv.boston <- cv.tree(tree.boston) #Cross-validation to find the optimal tree size.
# no FUN = prune.misclass because there's nothing to misclassify,
# instead you measure how far off the predicted values are
plot(cv.boston$size , cv.boston$dev,type ="b")

```



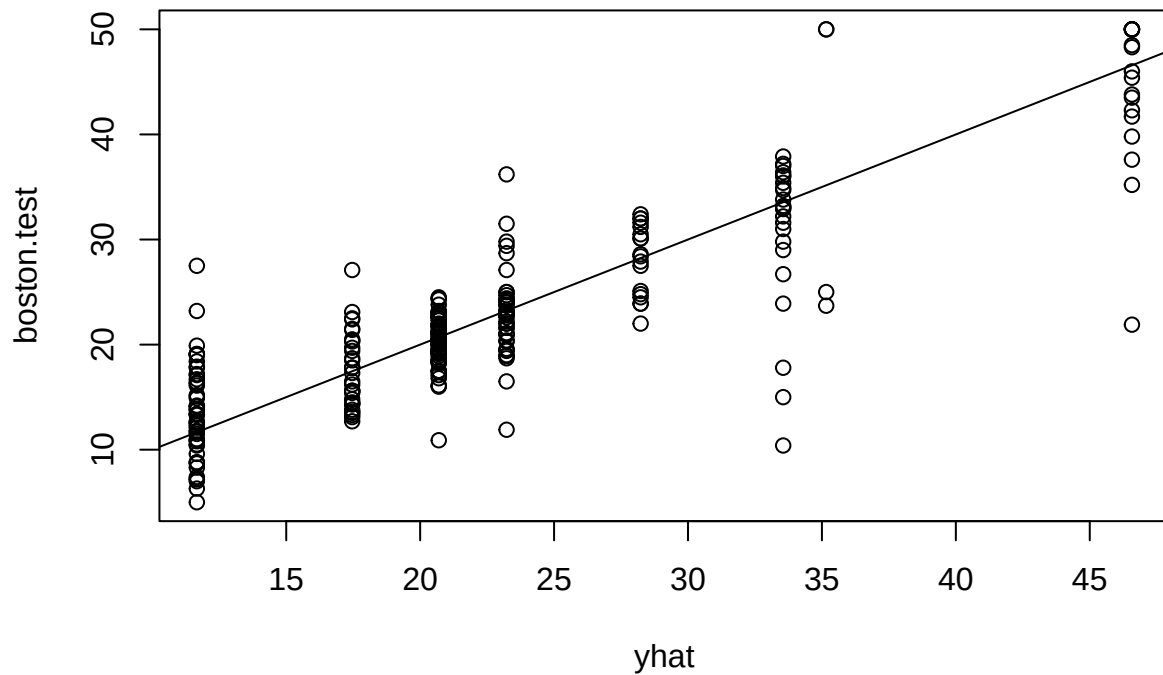
```
prune.boston <- prune.tree(tree.boston , best = 5) # optimal tree size approx. 5.  
# again: prune.tree bc of regression, there are no observations that are misclassified  
plot(prune.boston) # tree with 5 nodes instead of 8 - simpler and more interpretable  
text(prune.boston , pretty =0)
```

```

set.seed(123)
yhat <- predict(tree.boston , newdata = Boston[-train , ])
#Predicts medv for the test set sing the unpruned tree
boston.test <- Boston[-train,"medv"] #Extracts the actual medv values for the test set
plot(yhat , boston.test) #Plots predicted vs actual values
abline(0, 1)

```



```
#draws a 45-degree line. A perfect model would have all points sitting exactly on that line.
# more scatter -> worse predictions
mean((yhat - boston.test)^2) # 23.51
```

```
## [1] 23.51253
```

```
sqrt(23.51) #4.85 -> in thousands: model's predictions are off by roughly $4,850 on average.
```

```
## [1] 4.848711
```

Bagging and Random Forest

Bagging

Here we apply bagging and random forests to the Boston data, using the `randomForest` package in R.

```
library(randomForest)
```

```
## randomForest 4.7-1.2
```

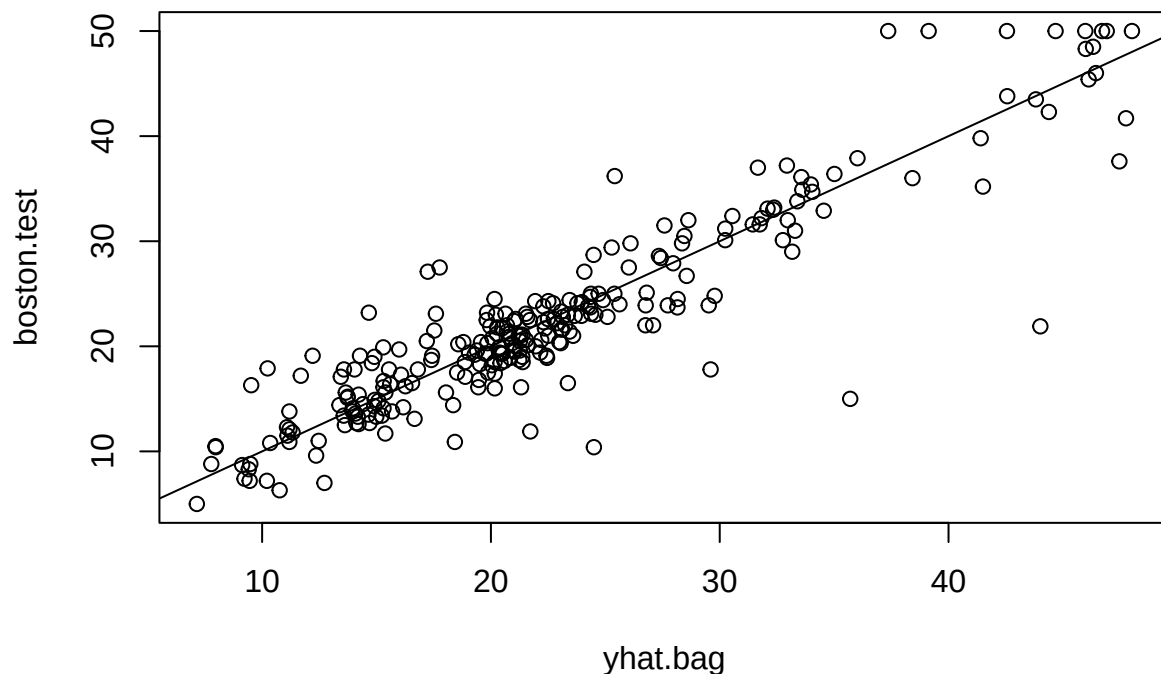
```
## Type rfNews() to see new features/changes/bug fixes.
```

```

set.seed(123)
bag.boston <- randomForest(medv ~ ., data = Boston, subset = train, mtry = 12, importance = TRUE)
# mtry = 12 indicates that all 12 predictors should be considered for each split
# of the tree, not just a subset (as in random forest)
# importance = TRUE : Tells R to track which variables mattered most across all trees.
# we can inspect this later with importance(bag.boston).

yhat.bag <- predict(bag.boston, newdata = Boston[-train,]) #predicts on test set
plot(yhat.bag, boston.test) #plots predicted vs actual medv
abline(0, 1)

```



```

mean((yhat.bag - boston.test)^2) # and compute test MSE

```

```
## [1] 14.63907
```

test set MSE associated with the bagged regression tree is 14.14, better than the single tree

```

bag.boston <- randomForest(medv ~ ., data = Boston, subset = train, mtry = 12, ntree = 25)
#same thing, but with 25 trees instead of default 500
yhat.bag <- predict(bag.boston, newdata = Boston[-train,])
# test set MSE
mean((yhat.bag - boston.test)^2) # 14.63 -> higher MSE (Fewer trees = faster but noisier)

```

```
## [1] 14.14494
```

Growing a random forest proceeds in exactly the same way, except that we use a smaller value of the `mtry` argument.

```
set.seed(123)
rf.boston <- randomForest(medv ~ ., data = Boston, subset = train, mtry = 6, importance = TRUE)
yhat.rf <- predict(rf.boston, newdata = Boston[-train,])
mean((yhat.rf - boston.test)^2)
```

```
## [1] 13.11186
```

```
# an improvement over bagging
```

Boosting

Here we use the `gbm()` function to fit boosted regression trees to the `Boston` data set

In boosting, each tree is grown from the mistakes of the previous tree. Specifically:

Tree 1 fits the data Tree 2 focuses on the observations Tree 1 got wrong Tree 3 focuses on what Tree 2 still got wrong ... and so on for 5000 trees

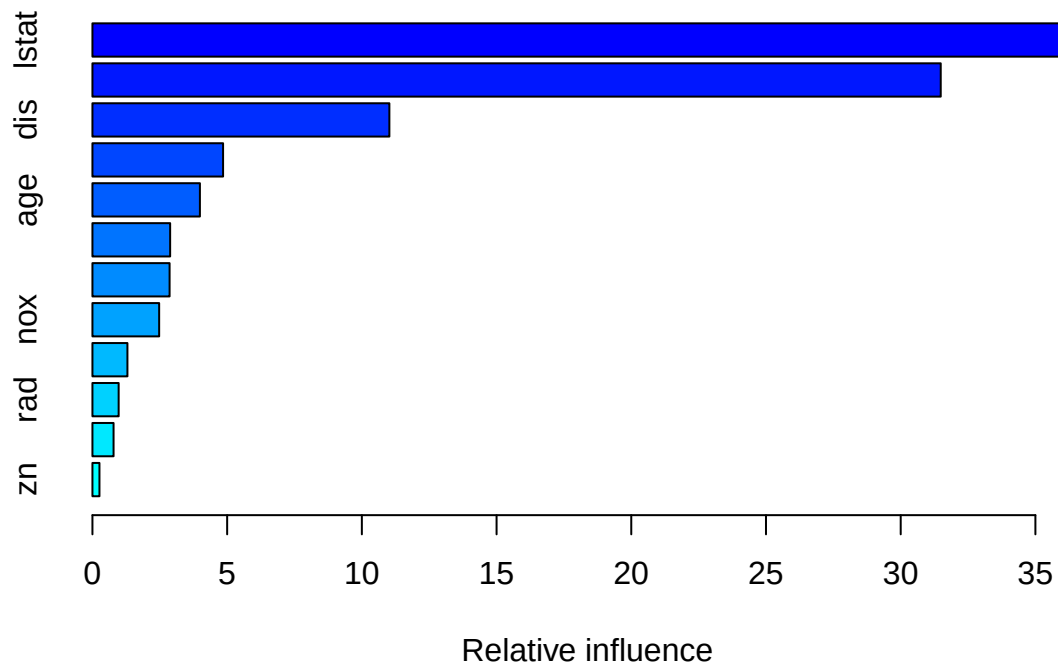
So the model learns gradually and corrects itself rather than building trees independently (as in bagging)

```
library(gbm)
```

```
## Loaded gbm 2.2.3
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
```

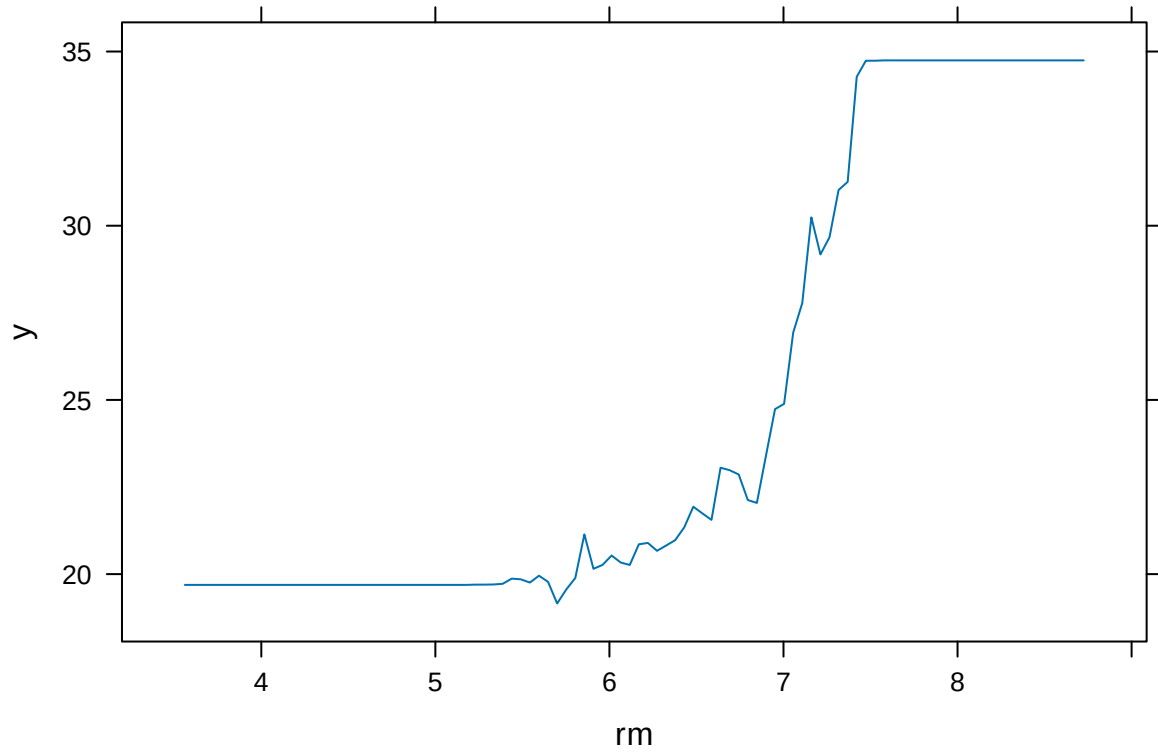
```
set.seed(123)
boost.boston <- gbm(medv ~ ., data = Boston[train, ],
  distribution = "gaussian", # since a regression problem
  n.trees = 5000,
  interaction.depth = 4 # each individual tree is kept shallow (max 4 splits)
)
summary(boost.boston)
```



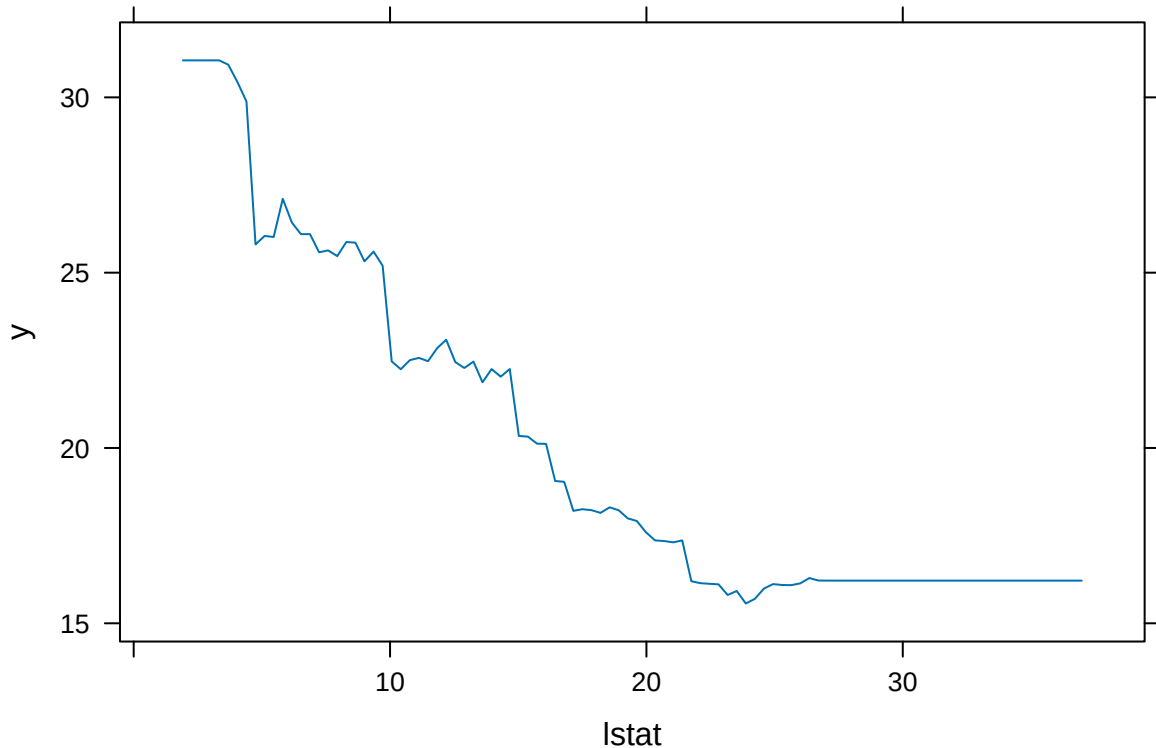
```
##      var    rel.inf
## lstat  lstat 37.1140525
## rm     rm   31.4862598
## dis    dis  11.0215024
## crim   crim  4.8494315
## age    age   3.9891630
## ptratio ptratio 2.8839523
## tax    tax   2.8633974
## nox    nox   2.4764052
## indus  indus  1.2987474
## rad    rad   0.9731853
## chas   chas  0.7834417
## zn     zn    0.2604613
```

```
# lstat and rm are the most important variables
```

```
# Produce partial dependence plots for these two variables
plot(boost.boston , i = "rm")
```



```
plot(boost.boston , i = "lstat")
```



These are partial dependence plots. They show the marginal effect of one variable on medv, after averaging out all other variables.

So for rm (avg rooms per house):

Houses with fewer than 5.5 rooms → predicted value flat around \$20k
Between 5.5 and 7.5 rooms → steep increase
Above 7.5 rooms → flattens out around \$35k

For lstat (% lower status population):

Low lstat → high house values (\$30k)
As lstat increases, values drop steeply
Above around 25% → flattens out around \$16k

The interesting thing is the non-linear shape. A linear regression would only capture a straight line relationship. The boosted tree captures these plateaus and steep sections automatically.

We now use the boosted model to predict medv on the test set.

```
set.seed(123)
yhat.boost <- predict(boost.boston,
                      newdata = Boston[-train , ], n.tree = 5000)
mean((yhat.boost - boston.test)^2)
```

```
## [1] 14.56972
```

```
# better than the test MSE of bagging
```

Perform boosting with a different value of the shrinkage parameter λ .

```
set.seed(123)
boost.boston <-gbm(medv ~ ., data = Boston[train , ], distribution = "gaussian",
                  n.trees = 5000, interaction.depth = 4, shrinkage =0.02, verbose =F)
yhat.boost <- predict(boost.boston, newdata = Boston[-train , ], n.trees = 5000)
mean((yhat.boost - boston.test)^2)
```

```
## [1] 15.0413
```